

Helix Chat Context Engine

How each editor-chat turn is assembled to minimize tokens and latency — fully deterministic, zero extra AI calls. Shipped 2026-06-12 (commit 73d2e5a).

1 · Request

POST /api/workspaces/[id]/chat
auth guard · rate limit · guest meter
chat/route.ts

2 · Raw inputs

- file tree (1 fetch)
- package.json
- last 40 messages + stored tool actions
- Workspace.notes



3 · Context engine — src/lib/chat-context.ts (pure functions, no AI)

stackLine()

Detects the stack from package.json
"Stack: Next.js + tailwindcss"

~10 tokens

treeOutline()

Dirs collapsed to counts, entry files surfaced — replaces 200-path dump
src/components/ (14 files)

~30–300 tokens

historyContext()

Older turns → 1-line digest each (uses stored tool labels) · last 8 messages stay verbatim
assistant: built navbar [wrote 2 file(s)]

digest ≤ 1.5k chars

PROJECT NOTES

Agent-curated memory via the **remember** tool — Workspace.notes, ≤2k chars, replace-on-write, invisible to users

⬅ durable memory

fitBudget() — guardrail

Hard ceiling 24,000 input chars. Trim order: digest → tree (depth 1) → verbatim window (min 4 msgs). Rules, notes & current message never trimmed.



4 · Agent loop (≤6 tool hops)

OpenAI / Anthropic / local

- list_files ← request-scoped tree cache (no refetch)
- read_file (on demand, ≤24k)
- write_files / delete_file (invalidate cache)
- remember (update notes)
workspace-tools.ts · ai-agent.ts

5 · Persist & respond

user + assistant messages, tool actions, honest token count (full-input fallback estimate)
→ change manifest updates the editor live

Before — turn 15, imported 100-file repo

Full path list every turn (≤200 paths) + 30 verbatim messages, no summarization, tree refetched per tool call.

≈ **9,000–10,500 input tokens**

guest allowance (15k) gone in ~1.5 turns

After — same turn, measured in verification

Stack line + tree outline + notes + digest + 8 verbatim messages, budget-capped.

≈ **900–1,800 input tokens (–80–90%)**

faster first token · ~10× more guest turns · zero added AI calls